🛡️ Module 11: Reverse Engineering, Fuzzing and Binary Exploitation

```
# Exercise 11: Performing Reverse Engineering with Debuggers
```

Scenario

Dynamic Binary Analysis with x32dbg allows reverse engineers and security analysts to inspect real-time CPU register states, stack pointer modifications, and memory allocation during runtime execution of Windows Portable Executables (**PE**).

Key dynamic debugging concepts with x32dbg:
1. **Breakpoint & Symbol Navigation**: Setting execution breakpoints at entry points (`PUSH EBP` at `0x00401310`), navigating to specific expressions (**Go To -> Expression**), and stepping through instructions (`Step Into` / `F7`).
2. **Stack Frame Verification**: Upon executing function prologue instructions (`PUSH EBP` followed by `MOV EBP, ESP`), the Base Pointer register (**EBP**) and Stack Pointer register (**ESP**) contain identical memory addresses before stack allocation (`SUB ESP, 0x38`).
3. **Runtime Register Auditing**: Observing live memory modifications, step-by-step register updates, string pointers, and conditional jump evaluations (`JZ` / `JNZ`).

In this exercise, we log into **Windows 11** (**Admin** / **Pa$$w0rd**), open **x32dbg.exe**, load target binary **crackme0x00.exe**, navigate to the main function address (`0x00401310`), set breakpoints, step through the function prologue, verify that **ESP** matches **EBP** after two steps, and step through runtime password verification logic.

Objective

The objective of this lab is to help students learn how to:
- Launch x32dbg debugger on Windows 11 target VM (**C:\Users\Admin\Downloads\Tools\x32\x32dbg.exe**)
- Load target 32-bit Portable Executable **crackme0x00.exe**
- Inspect entry point breakpoints (`PUSH EBP` at `0x00401260`)
- Navigate to Ghidra-verified main entry address **0x00401310** using **Go To -> Expression**
- Set toggle breakpoints (`F2`) at `0x00401310` and run execution (`F9`)
- Step into assembly instructions twice (`F7`) through `PUSH EBP` and `MOV EBP, ESP`
- Verify that register **ESP** holds the exact same memory address value as **EBP**
- Step through stack allocation (`SUB ESP, 0x38`), observe register updates, and add custom code comments

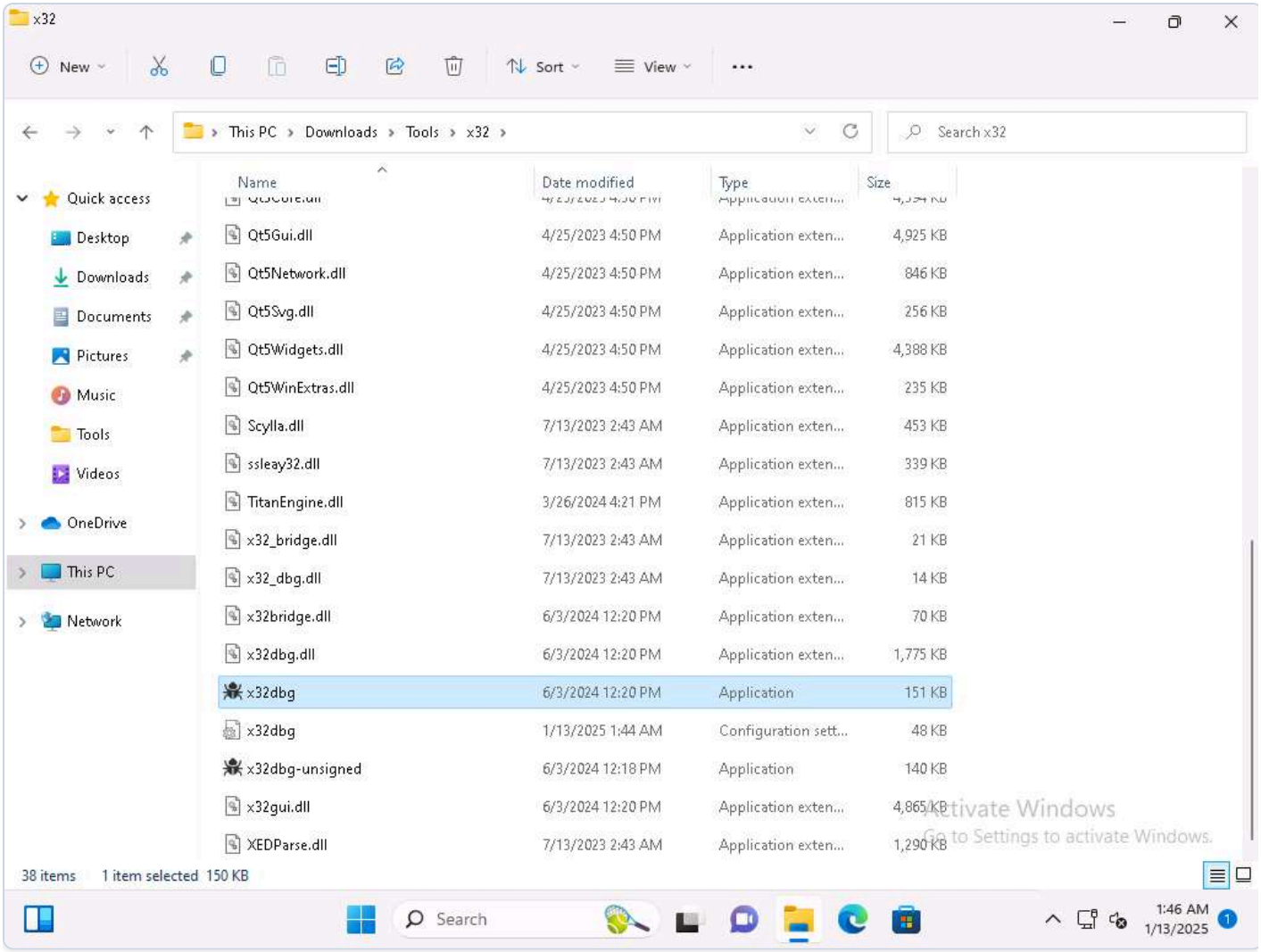
---

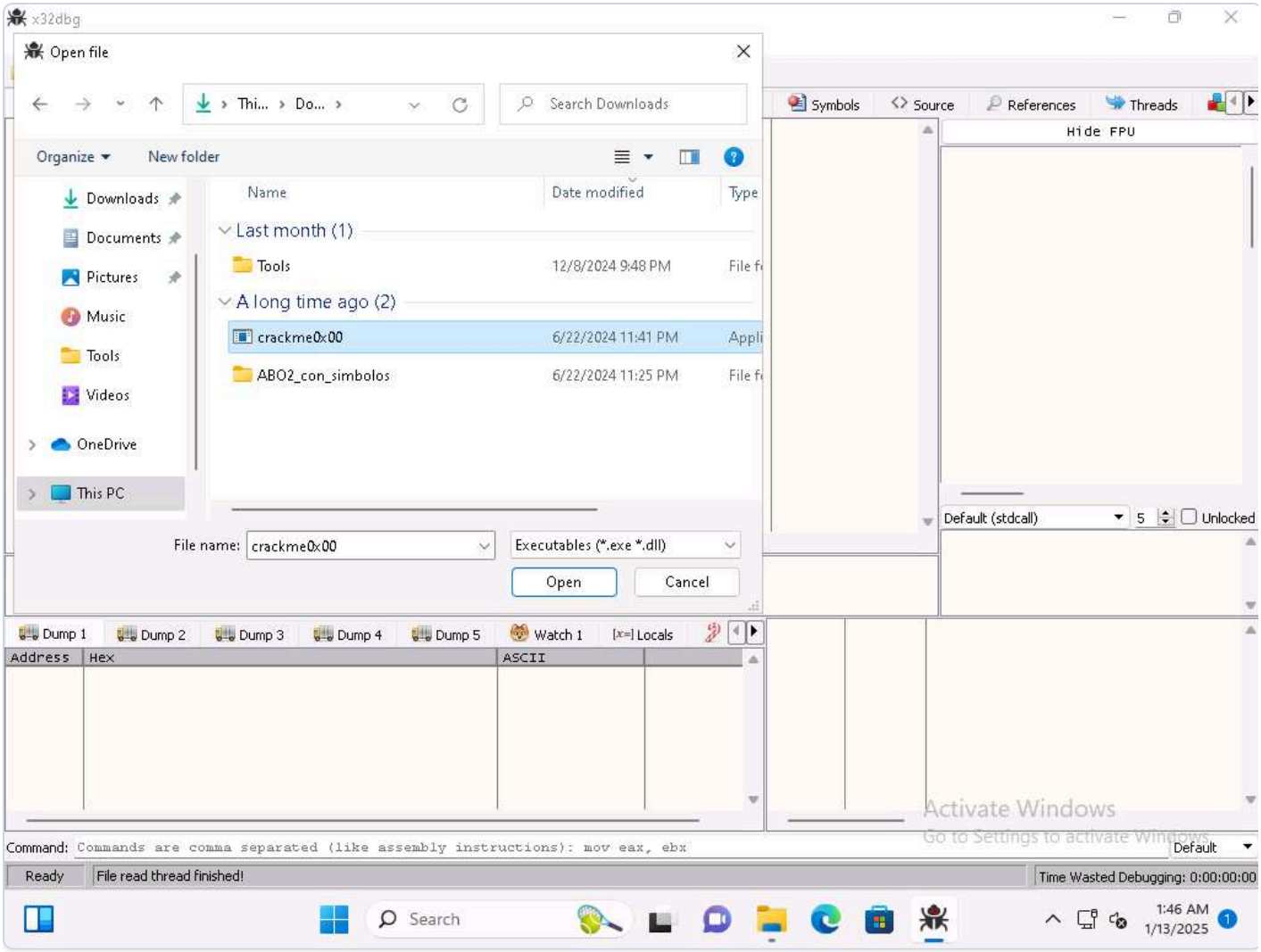
Task 1: Load crackme0x00.exe in x32dbg, Navigate to 0x00401310, and Inspect EBP/ESP Registers

In this task, we launch x32dbg on Windows 11, load `crackme0x00.exe` , navigate to main address `0x00401310` , toggle breakpoints, step into the prologue, verify `ESP` equals `EBP` , and step through dynamic execution.

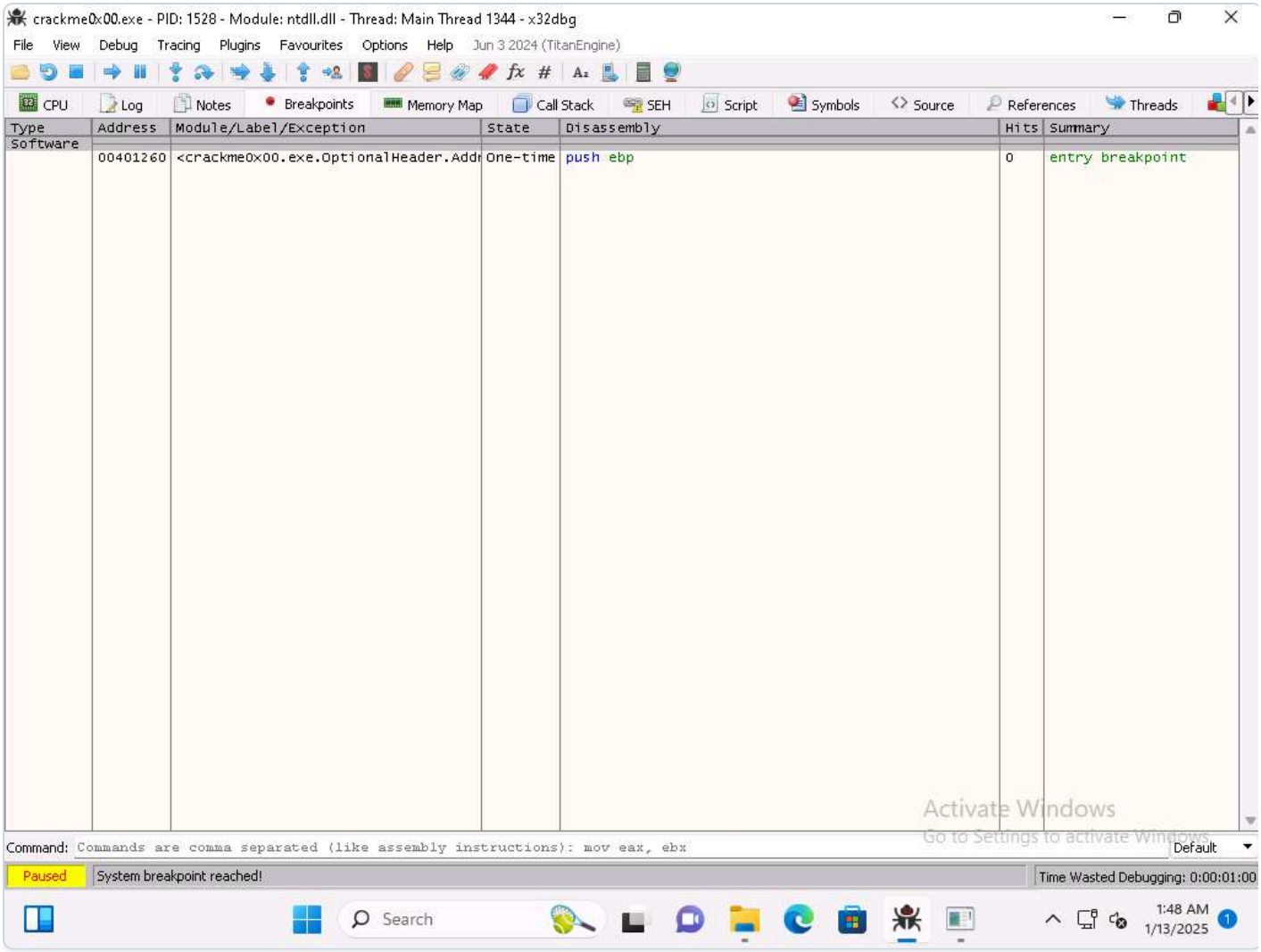
Lab Tasks

1. Switch to **Windows 11** VM, navigate to `C:\Users\Admin\Downloads\Tools\x32` , and double-click **x32dbg.exe**. Load **crackme0x00.exe** in x32dbg.

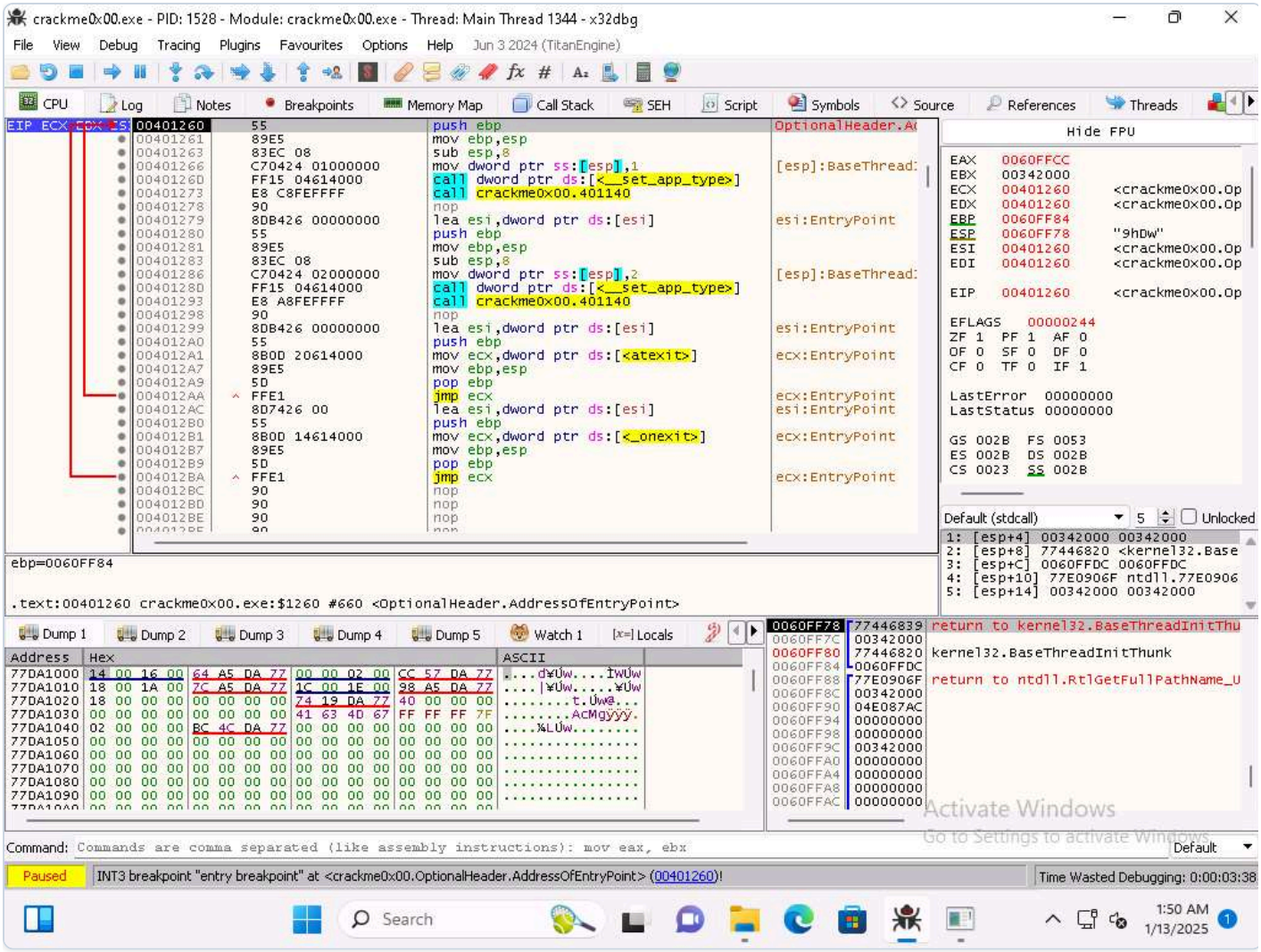

x32
New
Sort
View
This PC › Downloads › Tools › x32 ›
Search x32
Quick access
Desktop
Downloads
Documents
Pictures
Music
Tools
Videos
OneDrive
This PC
Network
Name
Date modified
Type
Size
Qt5Gui.dll 4/25/2023 4:50 PM Application exten... 4,925 KB
Qt5Network.dll 4/25/2023 4:50 PM Application exten... 846 KB
Qt5Svg.dll 4/25/2023 4:50 PM Application exten... 256 KB
Qt5Widgets.dll 4/25/2023 4:50 PM Application exten... 4,388 KB
Qt5WinExtras.dll 4/25/2023 4:50 PM Application exten... 235 KB
Scylla.dll 7/13/2023 2:43 AM Application exten... 453 KB
ssleay32.dll 7/13/2023 2:43 AM Application exten... 339 KB
TitanEngine.dll 3/26/2024 4:21 PM Application exten... 815 KB
x32_bridge.dll 7/13/2023 2:43 AM Application exten... 21 KB
x32_dbg.dll 7/13/2023 2:43 AM Application exten... 14 KB
x32bridge.dll 6/3/2024 12:20 PM Application exten... 70 KB
x32dbg.dll 6/3/2024 12:20 PM Application exten... 1,775 KB
x32dbg 6/3/2024 12:20 PM Application 151 KB
x32dbg 1/13/2025 1:44 AM Configuration sett... 48 KB
x32dbg-unsigned 6/3/2024 12:18 PM Application 140 KB
x32gui.dll 6/3/2024 12:20 PM Application exten... 4,865 KB
XEDParse.dll 7/13/2023 2:43 AM Application exten... 1,290 KB
Activate Windows
Go to Settings to activate Windows.
38 items 1 item selected 150 KB
Search
1:46 AM
1/13/2025



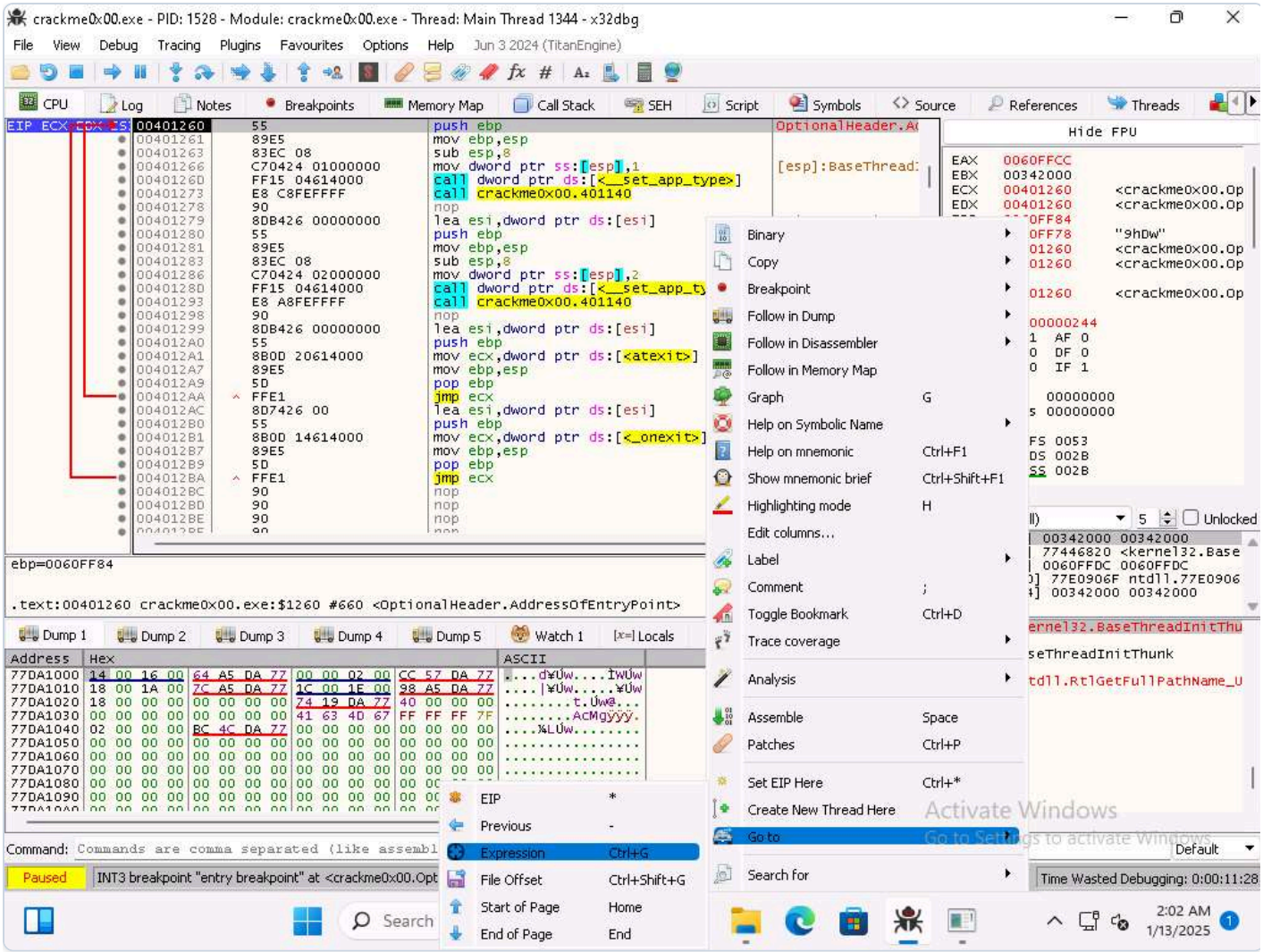
1. Select **Breakpoints** tab and verify default entry point breakpoint at `0x00401260`.

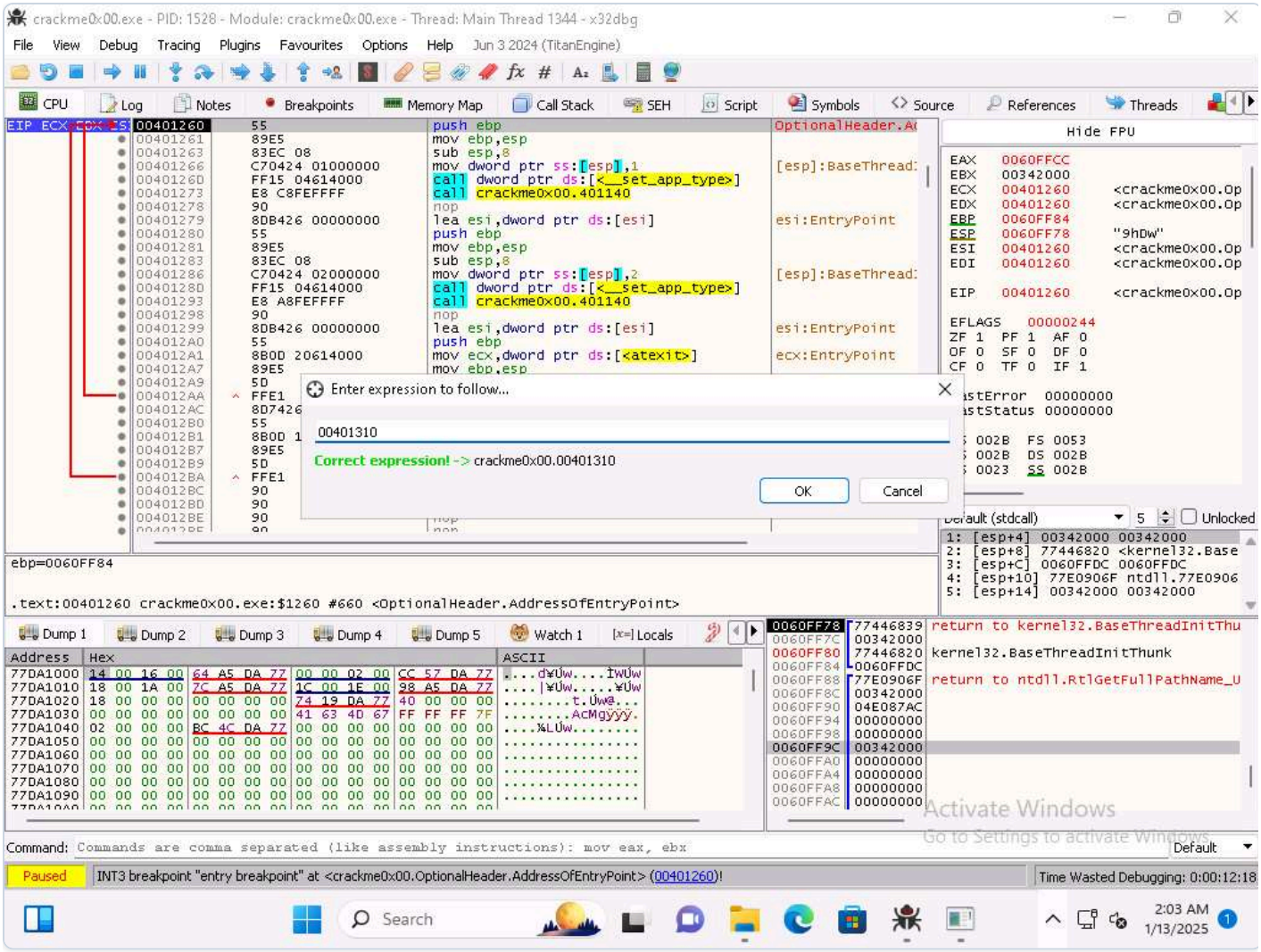


1. Click **Run** button (`F9`) to execute program to initial entry point (`0x00401260`).

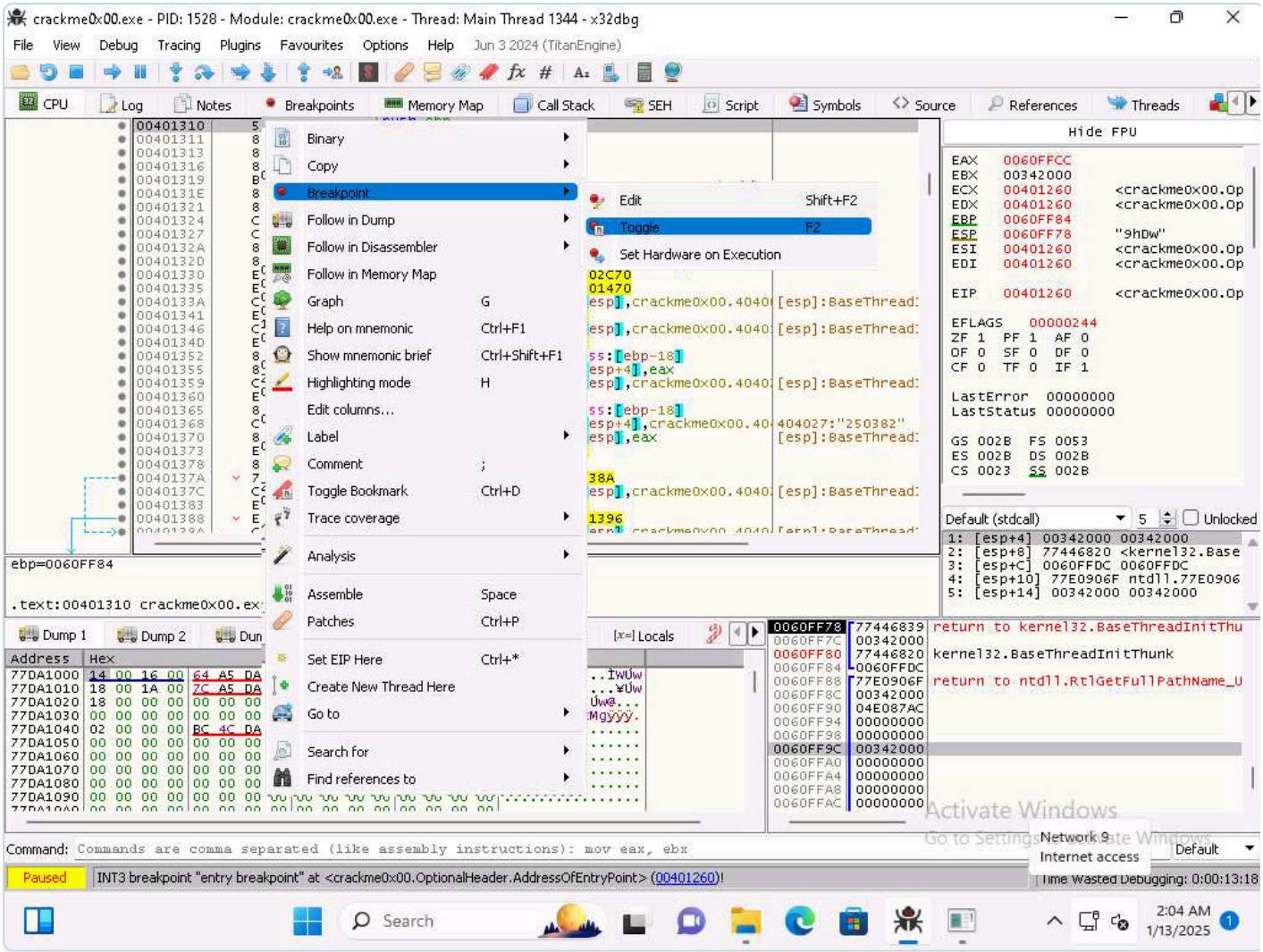


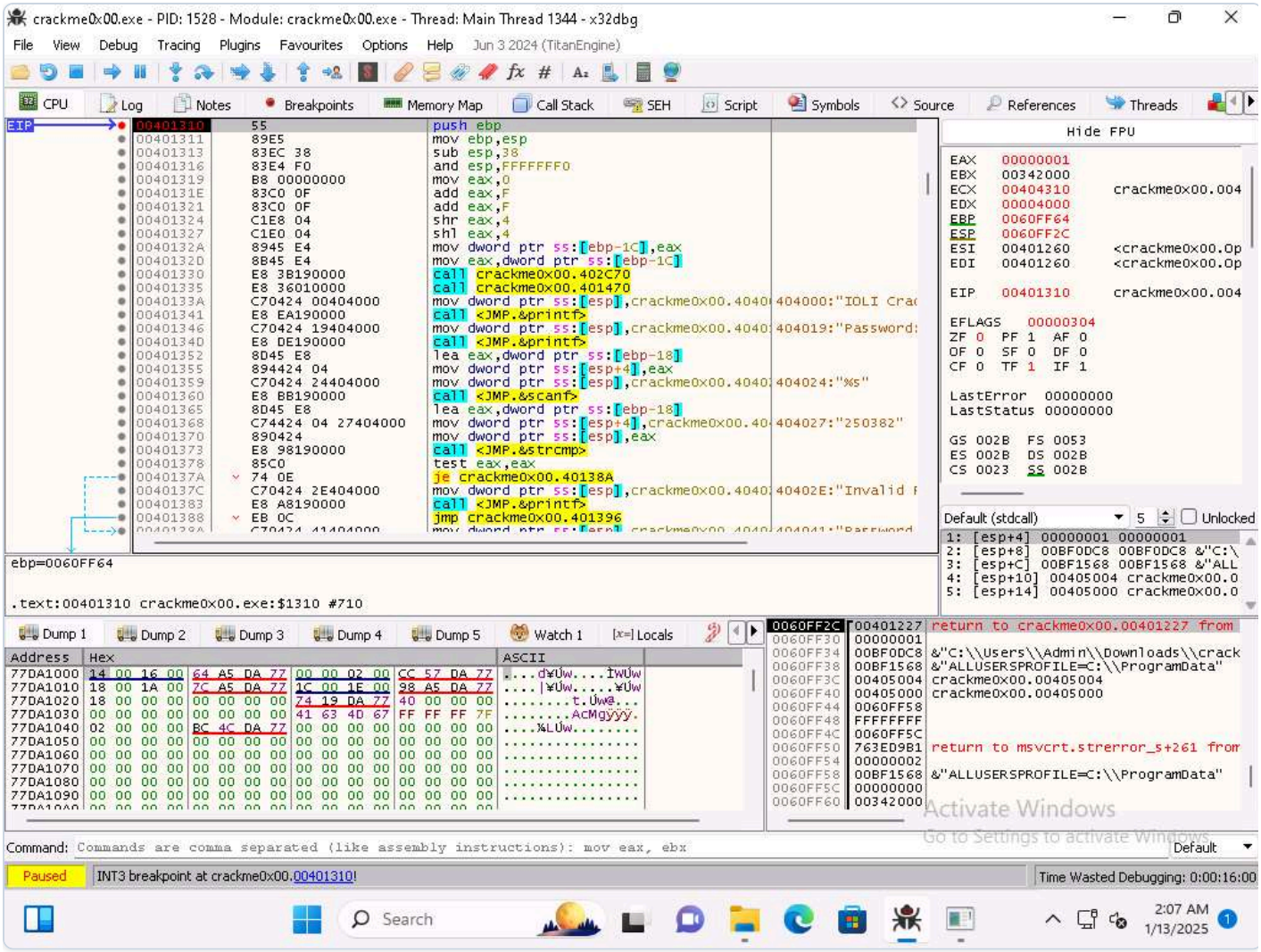
1. Right-click in CPU Disassembler pane, select **Go To -> Expression**, and enter address **0x00401310** (Ghidra main function start).


crackme0x00.exe - PID: 1528 - Module: crackme0x00.exe - Thread: Main Thread 1344 - x32dbg
File View Debug Tracing Plugins Favourites Options Help Jun 3 2024 (TitanEngine)
CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads
Hide FPU
Binary
Copy
Breakpoint
Follow in Dump
Follow in Disassembler
Follow in Memory Map
Graph G
Help on Symbolic Name
Help on mnemonic Ctrl+F1
Show mnemonic brief Ctrl+Shift+F1
Highlighting mode H
Edit columns...
Label
Comment ;
Toggle Bookmark Ctrl+D
Trace coverage
Analysis
Assemble Space
Patches Ctrl+P
Set EIP Here Ctrl+*
Create New Thread Here
Go to
Search for
EIP *
Previous -
Expression Ctrl+G
File Offset Ctrl+Shift+G
Start of Page Home
End of Page End
Activate Windows
Paused
INT3 breakpoint "entry breakpoint" at <crackme0x00.Opt
Time Wasted Debugging: 0:00:11:28
2:02 AM
1/13/2025

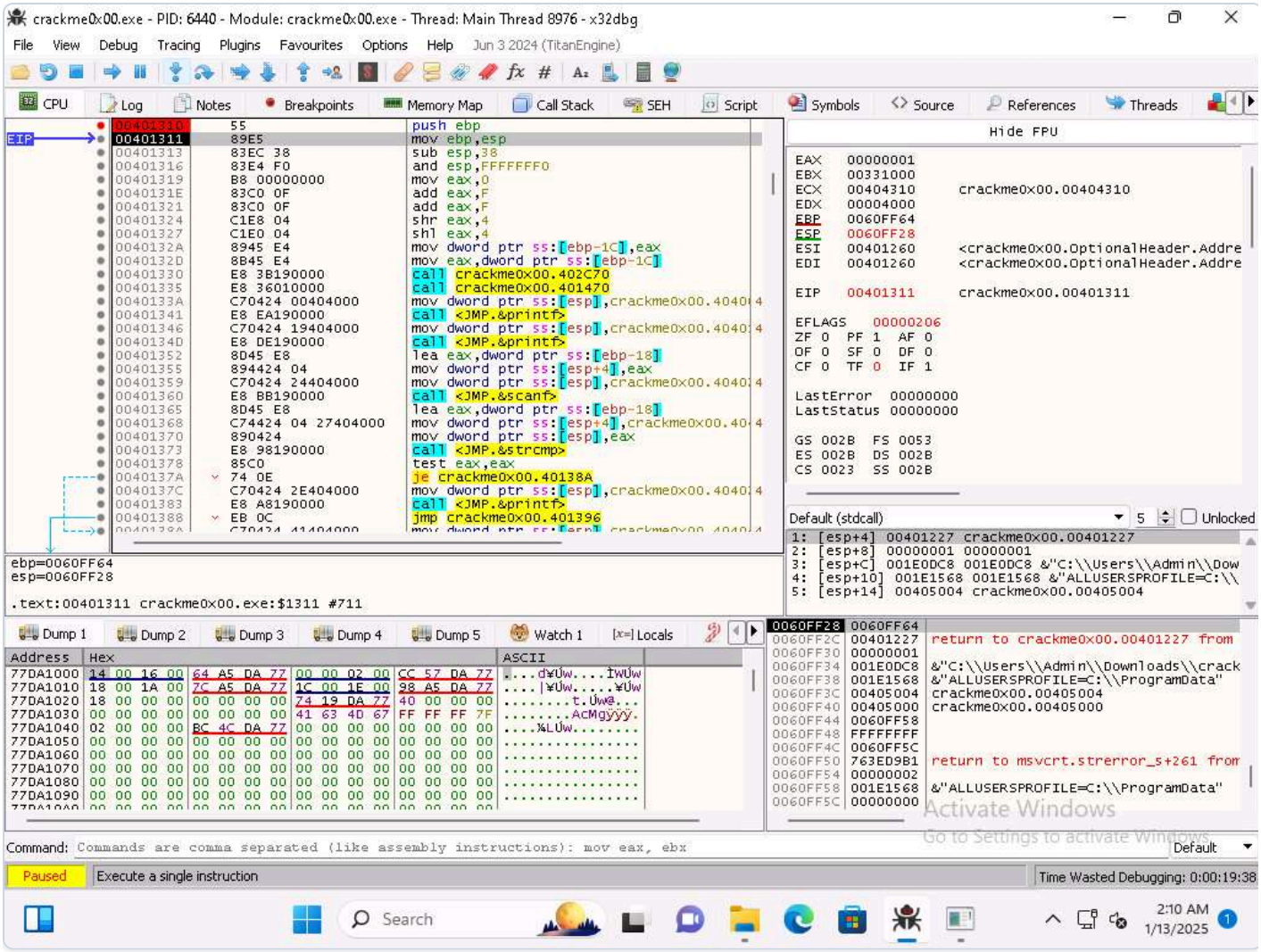


1. Toggle breakpoint at `0x00401310` (`F2`) and click **Run** (`F9`) to jump to main function breakpoint.

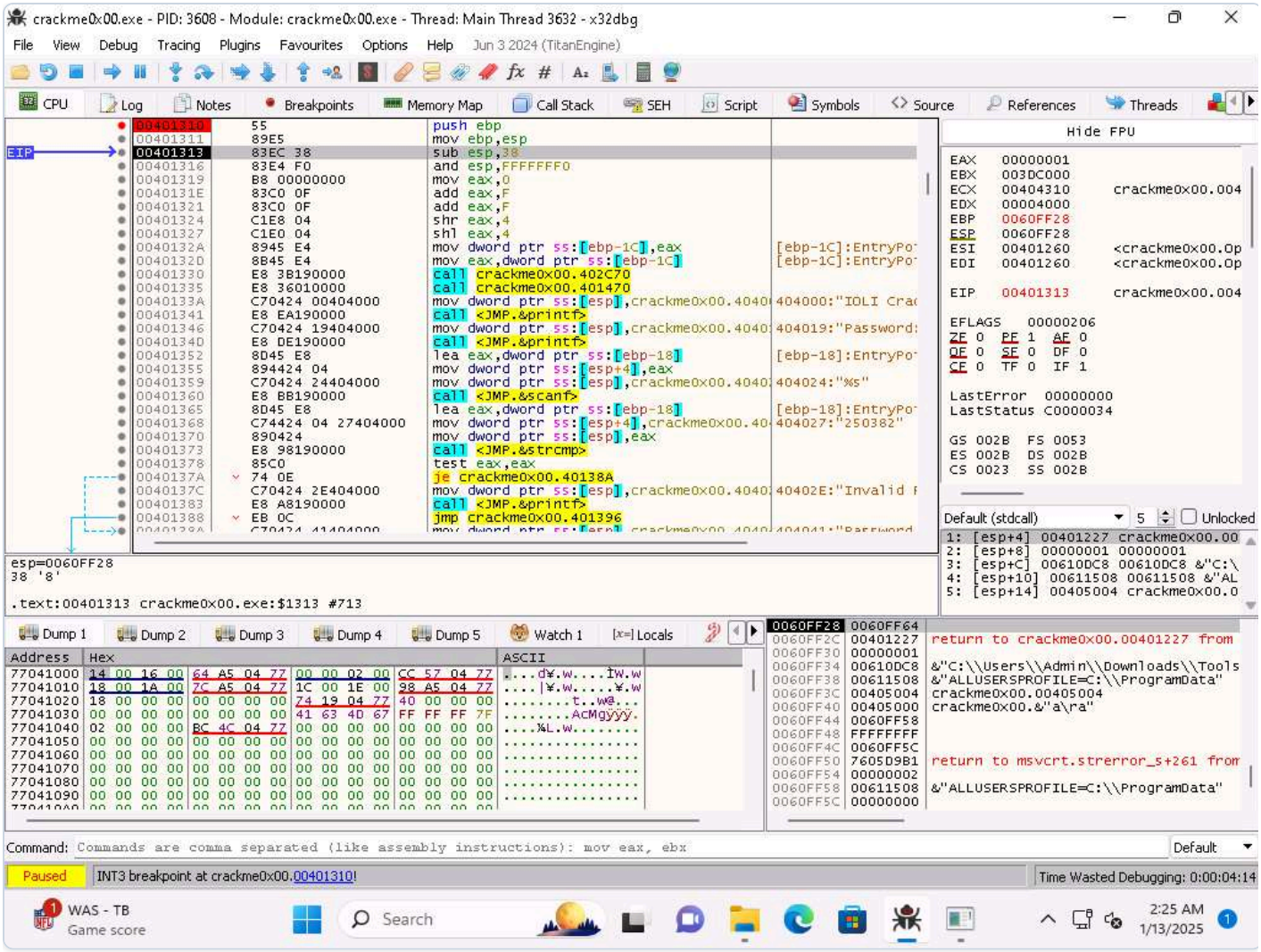

crackme0x00.exe - PID: 1528 - Module: crackme0x00.exe - Thread: Main Thread 1344 - x32dbg
File View Debug Tracing Plugins Favourites Options Help Jun 3 2024 (TitanEngine)
CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads
Binary
Copy
Breakpoint
Follow in Dump
Follow in Disassembler
Follow in Memory Map
Graph G
Help on mnemonic Ctrl+F1
Show mnemonic brief Ctrl+Shift+F1
Highlighting mode H
Edit columns...
Label
Comment ;
Toggle Bookmark Ctrl+D
Trace coverage
Analysis
Assemble Space
Patches Ctrl+P
Set EIP Here Ctrl+*
Create New Thread Here
Go to
Search for
Find references to
Edit Shift+F2
Toggle F2
Set Hardware on Execution
Hide FPU
EAX 0060FFCC
EBX 00342000
ECX 00401260 <crackme0x00.Op
EDX 00401260 <crackme0x00.Op
EBP 0060FF84
ESP 0060FF78 "9hDw"
ESI 00401260 <crackme0x00.Op
EDI 00401260 <crackme0x00.Op
EIP 00401260 <crackme0x00.Op
EFLAGS 00000244
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1
LastError 00000000
LastStatus 00000000
GS 002B FS 0053
ES 002B DS 002B
CS 0023 SS 002B
Default (stdcall) 5 Unlocked
1: [esp+4] 00342000 00342000
2: [esp+8] 77446820 <kernel32.Base
3: [esp+C] 0060FFDC 0060FFDC
4: [esp+10] 77E0906F ntdll.77E0906
5: [esp+14] 00342000 00342000
ebp=0060FF84
.text:00401310 crackme0x00.ex
Dump 1 Dump 2 Locals
Address Hex
77DA1000 14 00 16 00 64 A5 DA
77DA1010 18 00 1A 00 7C A5 DA
77DA1020 18 00 00 00 00 00 00
77DA1030 00 00 00 00 00 00 00
77DA1040 02 00 00 00 BC 4C DA
77DA1050 00 00 00 00 00 00 00
77DA1060 00 00 00 00 00 00 00
77DA1070 00 00 00 00 00 00 00
77DA1080 00 00 00 00 00 00 00
77DA1090 00 00 00 00 00 00 00
0060FF78 77446839 return to kernel32.BaseThreadInitThu
0060FF7C 00342000
0060FF80 77446820 kernel32.BaseThreadInitThunk
0060FF84 0060FFDC
0060FF88 77E0906F return to ntdll.RtlGetFullPathName_U
0060FF8C 00342000
0060FF90 04E087AC
0060FF94 00000000
0060FF98 00000000
0060FF9C 00342000
0060FFA0 00000000
0060FFA4 00000000
0060FFA8 00000000
0060FFAC 00000000
Activate Windows
Go to Settings to activate Windows.
Network 9
Internet access
Command: Commands are comma separated (like assembly instructions): mov eax, ebx
Default
Paused
INT3 breakpoint "entry breakpoint" at <crackme0x00.OptionalHeader.AddressOfEntryPoint> (00401260)!
Time Wasted Debugging: 0:00:13:18
Search
2:04 AM
1/13/2025



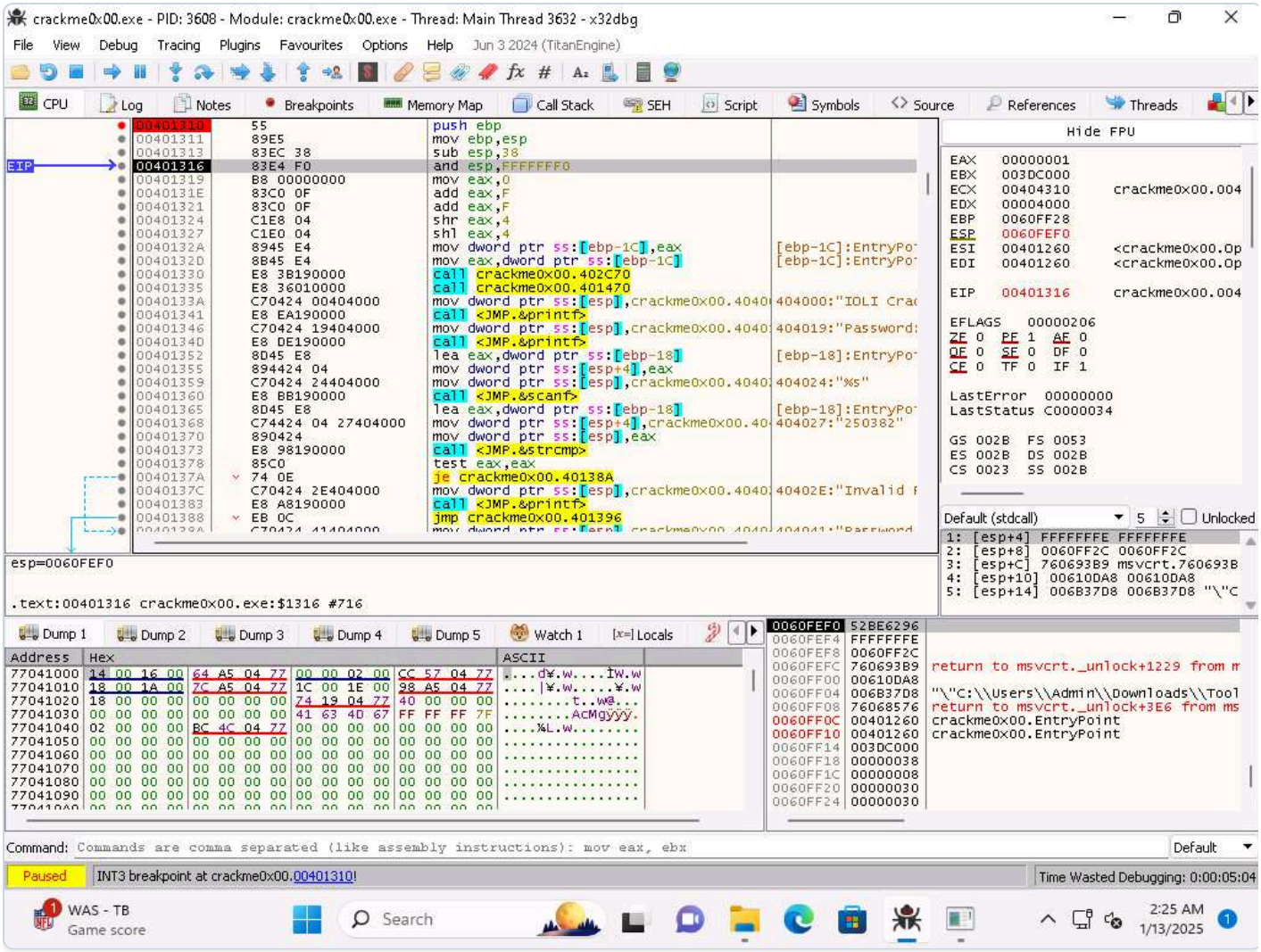
1. Step into instruction twice (`F7`) executing `PUSH EBP` and `MOV EBP, ESP`.



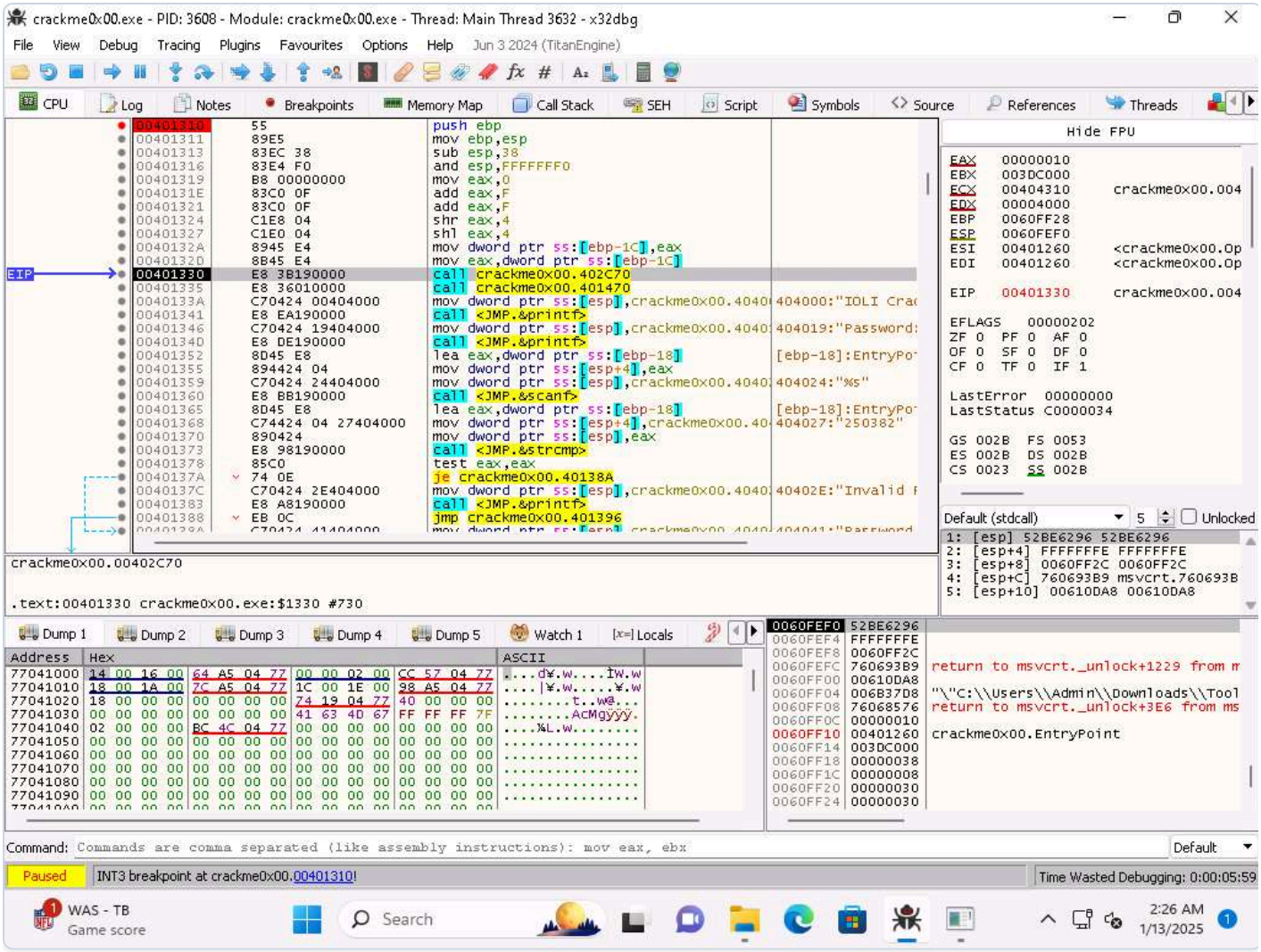
1. Reach `SUB ESP, 0x38` instruction. Inspect CPU Register pane and verify register **ESP** holds identical address value as **EBP**.



1. Step into `SUB ESP, 0x38` (`F7`) and observe `ESP` register value decrease by `0x38` bytes.



1. Continue stepping into execution (`F7`), observe compiler setup calls, add code comments, and inspect password prompt string processing.



This concludes the lab exercise.

IMPORTANT: ❓ **Question 11.11.1.1** On a Windows 11 system, utilize reverse engineering techniques with the x32dbg debugger to analyze and understand the functionality of the crackme0x00 binary application. In this lab, identify which register holds the same value as EBP after executing two steps in the code. [!SUCCESS] ✅

Answer: **esp**